

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 3 : Utilisation des opérateurs mathématiques

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

3.0 Introduction au Chapitre 3

Les mathématiques dans le monde physique

L'Arduino interagit avec le monde réel via des tensions et des signaux. Transformer ces signaux bruts en informations utiles (température, distance, vitesse) nécessite des calculs mathématiques, allant de la simple addition à la trigonométrie et la manipulation binaire.

Objectifs de ce chapitre :

- Maîtriser l'arithmétique de base et ses pièges (division entière).
- Contraindre, borner et formater des données issues de capteurs.
- Générer des nombres aléatoires pour des comportements imprévisibles.
- Manipuler les données au niveau le plus bas : les bits et les octets.

3.1 Addition, Soustraction, Multiplication et Division

Problème : Réaliser des calculs de base sans perdre en précision.

Solution : Utiliser les opérateurs +, -, *, / en faisant attention aux types de variables.

```
int a = 5;
int b = 2;
int resultatInt = a / b;      // resultatInt vaut 2 ! (La decimale est tronquee)

float c = 5.0;
float d = 2.0;
float resultatFloat = c / d; // resultatFloat vaut 2.5
```

Le piège de la division entière

En C++, si vous divisez deux entiers, le résultat est un entier. Pour obtenir un résultat décimal, au moins l'un des nombres doit être un float (ex : a / 2.0;).

3.2 Incrémentation et Décrémentation

Problème : Ajouter ou soustraire rapidement une valeur (très fréquent dans les boucles).

Solution : Utiliser les opérateurs combinés.

```
int compteur = 0;

// Methode classique :
compteur = compteur + 1;

// Operateurs raccourcis (Compound operators) :
compteur++; // Ajoute 1 (Incrementation)
compteur--; // Soustrait 1 (Decrementation)
compteur += 5; // Ajoute 5 a la valeur actuelle
compteur *= 2; // Multiplie la valeur actuelle par 2
```

Note : Ces opérateurs raccourcis rendent le code plus lisible et sont souvent optimisés par le compilateur.

3.3 Trouver le reste d'une division (Modulo)

Problème : Créer un cycle répétitif ou vérifier si un nombre est pair.

Solution : Utiliser l'opérateur Modulo %.

```
int reste = 10 % 3; // reste vaut 1 (car 10 = 3*3 + 1)

// Exemple : Executer une action tous les 10 cycles
if (compteur % 10 == 0) {
    // Ce code s'exécute quand compteur vaut 0, 10, 20, 30...
    Serial.println("Dizaine atteinte !");
}
```

Figure 1: Le modulo est l'équivalent du reste dans la division euclidienne.

3.4 et 3.5 Valeur Absolue et Contrainte

Valeur Absolue : Retirer le signe négatif d'un nombre.

```
int ecart = abs(-50); // ecart vaut 50
```

Utile pour calculer la différence de tension entre deux capteurs sans se soucier de savoir lequel est le plus grand.

Contraindre une valeur (constrain) : **Problème :** Un capteur renvoie des valeurs aberrantes qui font planter le système.

Solution : Borner la variable entre un minimum et un maximum.

```
// constrain(valeur, min, max)
int securite = constrain(lectureCapteur, 0, 255);
// Si lectureCapteur = 300, securite vaudra 255.
// Si lectureCapteur = -10, securite vaudra 0.
```

3.6 Trouver le Minimum ou le Maximum

Problème : Comparer deux valeurs et conserver la plus extrême.

Solution : Utiliser `min()` et `max()`.

```
int valeurMax = max(capteur1, capteur2); // Garde la plus grande valeur
int valeurMin = min(100, lectureActuelle); // Ne dépassera jamais 100
```

Mise en garde (Margolis, p.74)

Évitez d'utiliser des fonctions mathématiques complexes ou des appels de fonctions à l'intérieur des parenthèses de `min()` et `max()` (ex : `max(a++, b)`). À cause de la façon dont ces macros sont définies en C++, cela peut causer des résultats imprévisibles.

3.7 et 3.8 Puissances et Racines Carrées

Calculer une puissance (pow) :

```
// pow(base, exposant) - Retourne un float  
float surface = PI * pow(rayon, 2); // Calcule l'aire d'un cercle
```

Calculer une racine carrée (sqrt) :

```
float hypotenuse = sqrt(pow(coteA, 2) + pow(coteB, 2)); // Theoreme de Pythagore
```

Attention : Ces fonctions sont très gourmandes en ressources processeur sur l'Arduino. Ne les utilisez que si c'est strictement nécessaire.

3.9 Arrondir des nombres à virgule flottante

Problème : Les calculs flottants génèrent des décimales indésirables.

Solution : Les fonctions d'arrondi C++.

- `round(x)` : Arrondit à l'entier le plus proche.
- `ceil(x)` : Arrondit toujours à l'entier supérieur (Plafond).
- `floor(x)` : Arrondit toujours à l'entier inférieur (Plancher).

```
float x = 3.6;  
int a = round(x); // a = 4  
int b = floor(x); // b = 3  
int c = ceil(x);  // c = 4
```

3.10 Utilisation des fonctions trigonométriques

Problème : Calculer des angles pour des bras robotiques ou des capteurs d'inclinaison.

Solution : `sin()`, `cos()`, et `tan()`.

Radians vs Degrés

En Arduino C++, les fonctions trigonométriques attendent des angles en **Radians**, et non en Degrés !

```
float angleDegrés = 90.0;
// Conversion Degrés -> Radians : (angle * PI) / 180
float angleRadians = (angleDegrés * PI) / 180.0;

float resultat = sin(angleRadians); // resultat = 1.0
```

3.11 Génération de Nombres Aléatoires

Problème : Créer des séquences imprévisibles (ex : un jeu de dés, des scintillements de LED).

Solution : La fonction `random()`.

```
long de = random(1, 7); // Genere un nombre entre 1 et 6 (le max est exclus)
```

Le problème de la "Graine" (Seed) : L'Arduino n'est pas capable de vrai hasard ("pseudo-random"). Si on ne l'aide pas, il générera toujours la même suite au démarrage.

```
void setup() {  
    // Initialise le generateur avec le bruit analogique d'une broche vide (A0)  
    randomSeed(analogRead(0));  
}
```

3.12 Manipuler les Bits (Bitwise operations)

Le niveau le plus bas de la machine. Un octet (byte) contient 8 bits (0 ou 1). L'Arduino propose des fonctions pour manipuler ces bits individuellement.

```
byte maVariable = 0b00001000; // Notation binaire (vaut 8 en decimal)

// Lire l'etat du bit numero 3 (les index commencent a 0, de droite a gauche)
int etat = bitRead(maVariable, 3); // etat vaut 1

// Mettre le bit 0 a 1 (Set)
bitSet(maVariable, 0); // maVariable devient 0b00001001 (9 en decimal)

// Mettre le bit 3 a 0 (Clear)
bitClear(maVariable, 3); // maVariable devient 0b00000001
```

3.13 Le décalage de Bits (Bit Shifting)

Problème : Déplacer des données binaires (très utilisé dans les communications SPI ou pour contrôler des registres à décalage).

Solution : Les opérateurs << (décalage à gauche) et >> (décalage à droite).

```
byte valeur = 0b00000011; // Vaut 3

// Decalage de 2 positions a gauche
byte resultat = valeur << 2; // Devient 0b00001100 (Vaut 12)
```

Astuce Mathématique : Un décalage à gauche d'une position (<< 1) équivaut à multiplier par 2. Un décalage à droite (>> 1) équivaut à diviser par 2. C'est beaucoup plus rapide pour le processeur qu'une multiplication standard !

3.14 Extraire les octets d'un nombre plus grand

Problème : Vous devez envoyer un `int` (16 bits) via une connexion série (qui n'accepte que des paquets de 8 bits / 1 octet).

Solution : Séparer l'entier en deux parties (High Byte et Low Byte).

```
int capteur = 1000; // Valeur sur 16 bits (0b00000011 11101000)

byte partieHaute = highByte(capteur); // Recupere 0b00000011
byte partieBasse = lowByte(capteur);  // Recupere 0b11101000

Serial.write(partieHaute); // Envoi du premier octet
Serial.write(partieBasse); // Envoi du second octet
```

3.15 Recomposer un nombre à partir d'octets

Problème : À la réception, vous avez deux byte isolés et vous voulez reconstituer le int d'origine (16 bits).

Solution : La fonction `word()` ou les opérations binaires combinées.

```
byte recuHaut = 0b00000011;
byte recuBas = 0b11101000;

// Methode simple (Arduino) :
int valeurReconstituee = word(recuHaut, recuBas); // Vaut 1000

// Methode experte (C++) avec decalage et OU binaire :
int valeurExpert = (recuHaut << 8) | recuBas;
```

Figure 2: Schéma de la reconstitution d'un mot de 16 bits.

- **Type des variables** : Surveillez vos types lors des divisions. Un calcul d'entiers produira toujours un entier.
- **Sécurisation** : Utilisez `constrain()` avant d'injecter une valeur de capteur dans un actionneur (comme un servomoteur ou une PWM) pour éviter les crashes matériels.
- **Hasard maîtrisé** : N'oubliez jamais `randomSeed(analogRead(0))` pour initialiser votre générateur aléatoire.
- **Performance** : La manipulation de bits (`bitRead`, bit shifting) est le moyen le plus rapide d'interagir avec le matériel et les protocoles de communication.

Fin du Chapitre 3. Le prochain chapitre abordera la Communication Série : envoyer et recevoir des données avec le monde extérieur.